



IUS
INSTITUT
UNIVERSITAIRE
DES SCIENCES

Université: Institut Universitaire des Sciences (IUS)

TD N⁰ 5: Modélisation Mathématiques

Nom & Prénom: SAINT-JEAN Marc-Evenort

Professeur: Ismael SAINT AMOUR

Niveau: L3

Date: Juin / 2026

TD1 — Analyse du réseau routier de Cap-Haïtien avec OpenStreetMap.

a) J'ai installé les bibliothèques de cartographie

```
python -m pip install osmnx matplotlib networkx
```

```
Administrator: Windows PowerShell
Collecting geopandas>=1.0.1 (from osmnx)
  Downloading geopandas-1.1.3-py3-none-any.whl.metadata (2.3 kB)
Requirement already satisfied: numpy>=1.24 in c:\users\smarc\appdata\local\programs\python\python314\lib\site-packages (from osmnx) (2.4.3)
Requirement already satisfied: pandas>=1.5 in c:\users\smarc\appdata\local\programs\python\python314\lib\site-packages (from osmnx) (3.0.1)
Requirement already satisfied: requests>=2.30 in c:\users\smarc\appdata\local\programs\python\python314\lib\site-packages (from osmnx) (2.32.5)
Collecting shapely>=2.0 (from osmnx)
  Downloading shapely-2.1.2-cp314-cp314-win_amd64.whl.metadata (7.1 kB)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\smarc\appdata\local\programs\python\python314\lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\smarc\appdata\local\programs\python\python314\lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\smarc\appdata\local\programs\python\python314\lib\site-packages (from matplotlib) (4.62.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\smarc\appdata\local\programs\python\python314\lib\site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in c:\users\smarc\appdata\local\programs\python\python314\lib\site-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in c:\users\smarc\appdata\local\programs\python\python314\lib\site-packages (from matplotlib) (12.1.1)
Requirement already satisfied: pyparsing>=3 in c:\users\smarc\appdata\local\programs\python\python314\lib\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\smarc\appdata\local\programs\python\python314\lib\site-packages (from matplotlib) (2.9.0.post0)
Collecting pyproj>=0.7.2 (from geopandas==1.0.1->osmnx)
  Downloading pyproj-0.12.1-cp314-cp314-win_amd64.whl.metadata (6.0 kB)
Collecting pyproj>=3.5.0 (from geopandas==1.0.1->osmnx)
  Downloading pyproj-3.7.2-cp314-cp314-win_amd64.whl.metadata (31 kB)
Requirement already satisfied: tzdata in c:\users\smarc\appdata\local\programs\python\python314\lib\site-packages (from pandas==1.5->osmnx) (2025.3)
Requirement already satisfied: certifi in c:\users\smarc\appdata\local\programs\python\python314\lib\site-packages (from pyproj==0.7.2->geopandas==1.0.1->osmnx) (2026.2.25)
Requirement already satisfied: six>=1.5 in c:\users\smarc\appdata\local\programs\python\python314\lib\site-packages (from python-dateutil==2.7->matplotlib) (1.17.0)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\smarc\appdata\local\programs\python\python314\lib\site-packages (from requests==2.30->osmnx) (3.4.5)
Requirement already satisfied: idna<4,>=2.5 in c:\users\smarc\appdata\local\programs\python\python314\lib\site-packages (from requests==2.30->osmnx) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\smarc\appdata\local\programs\python\python314\lib\site-packages (from requests==2.30->osmnx) (2.6.3)
Downloading osmnx-2.1.0-py3-none-any.whl (104 kB)
Downloading geopandas-1.1.3-py3-none-any.whl (342 kB)
Downloading pyproj-0.12.1-cp314-cp314-win_amd64.whl (23.6 MB)
----- 1.87 MB 40.1 MB/s 0:00:32
Downloading pyproj-3.7.2-cp314-cp314-win_amd64.whl (6.4 MB)
----- 1.87 MB 40.1 MB/s 0:01:20
Downloading shapely-2.1.2-cp314-cp314-win_amd64.whl (1.8 MB)
----- 1.87 MB 40.1 MB/s 0:00:00
Installing collected packages: shapely, pyproj, pyproj, geopandas, osmnx
Successfully installed geopandas-1.1.3 osmnx-2.1.0 pyproj-0.12.1 pyproj-3.7.2 shapely-2.1.2

[notice] A new release of pip is available: 25.3 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\windows\system32>
```

```
File Edit Selection View Go Run ...
C:\> Users > smarc > OneDrive > Documents > IUS.Doc26 > Modelisation Maths > TD1_Reseau_Routier_CapHaïtien.py > ...

1 import osmnx as ox
2 import networkx as nx
3 import matplotlib.pyplot as plt
4 import matplotlib.cm as cm
5 import matplotlib.colors as colors
6
7 print("="*60)
8 print("TD1 — ANALYSE DU RÉSEAU ROUTIER DE CAP-HAÏTIEN (OSM)")
9 print("="*60)
10
11 # 1. CHARGER LA CARTE
12 print("📄 Téléchargement du réseau routier de Cap-Haïtien via coordonnées...")
13 # Coordonnées du centre-ville du Cap-Haïtien avec un rayon de 2.5 km
14 G_raw = ox.graph_from_point((19.7500, -72.2000), dist=2500, network_type='drive')
15
16 # 2. NETTOYER LE GRAPHES
17 print("🧹 Nettoyage et simplification du réseau...")
18 strongly_connected_nodes = max(nx.strongly_connected_components(G_raw), key=len)
19 G = G_raw.subgraph(strongly_connected_nodes).copy()
20
21 # 3. CALCULER LES MÉTRIQUES
22 print("📊 Calcul des métriques du réseau...")
23 nombre_noeuds = G.number_of_nodes()
24 nombre_routes = G.number_of_edges()
25 densite = nx.density(G)
26
27 print(f"• Nombre de carrefours (Nœuds) : {nombre_noeuds}")
28 print(f"• Nombre de segments de route (Arrêtes) : {nombre_routes}")
29 print(f"• Densité du réseau : {densite:.4f}")
30
31 # 4. IDENTIFIER LES POINTS CRITIQUES (Centralité)
32 print("👑 Identification des carrefours critiques...")
33
Ln 63, Col 11 Spaces: 4 UTF-8 CRLF {} Python Signed out Python 3.14.3 0-0 Go Live
12:42 AM 6/9/2026
```

```
33 centralite = nx.betweenness_centrality(G, weight='length')
34 noeud_critique = max(centralite, key=centralite.get)
35 print(f"    ID du carrefour le plus critique : {noeud_critique}")
36
37 # Ajustement de la taille des nœuds
38 node_sizes = [10 + (centralite[node] * 2000) for node in G.nodes()]
39
40 # CORRECTION OSMNX 2.0+ : Génération manuelle de la carte de couleur (cmap)
41 cmap = cm.get_cmap('hot')
42 norm = colors.Normalize(vmin=min(centralite.values()), vmax=max(centralite.values()))
43 node_colors = [cmap(norm(centralite[node])) for node in G.nodes()]
44
45 # 5. VISUALISATION AVANCÉE
46 print("🔥 Génération de la carte finale...")
47 fig, ax = ox.plot_graph(
48     G,
49     node_color=node_colors,
50     node_size=node_sizes,
51     edge_color='k', # Routes en gris foncé
52     edge_linewidth=1.2,
53     bgcolor='k', # Fond noir style "Dark Mode"
54     show=False,
55     close=False
56 )
57
58 ax.set_title("Analyse du Réseau Routier - Cap-Haïtien\n(Points chauds = Carrefours critiques)",
59             color='white', fontsize=14, fontweight='bold', pad=10)
60
61 print(f"🔥")
62 print("Tout fonctionne ! Le graphique va s'afficher...")
63 plt.show()
```

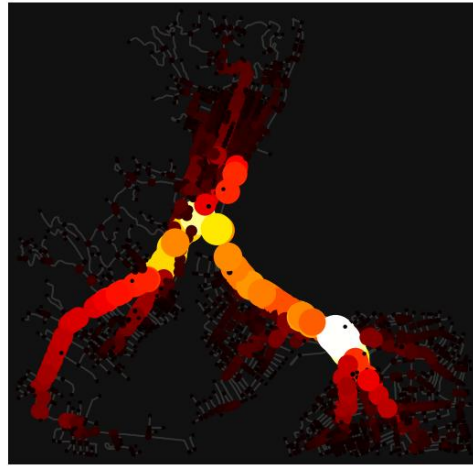
Modélisation réussie du réseau routier de Cap-Haïtien avec OSMnx et NetworkX. L'analyse a permis de calculer les métriques et de cartographier précisément les carrefours critiques

```
51 edge_color='k', # Routes en gris foncé
52 edge_linewidth=1.2,
53 bgcolor='k', # Fond noir style "Dark Mode"
54 show=False,
55 close=False
56 )
57
58 ax.set_title("Analyse du Réseau Routier - Cap-Haïtien\n(Points chauds = Carrefours critiques)",
59             color='white', fontsize=14, fontweight='bold', pad=10)
60
61 print(f"🔥")
62 print("Tout fonctionne ! Le graphique va s'afficher...")
63 plt.show()
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Téléchargement du réseau routier de Cap-Haïtien via coordonnées...
Nettoyage et simplification du réseau...
Calcul des métriques du réseau...
• Nombre de carrefours (Nœuds) : 1664
• Nombre de segments de route (Arrêtes) : 4706
• Densité du réseau : 0.0017
Identification des carrefours critiques...
• ID du carrefour le plus critique : 620039877
c:\Users\smarc\OneDrive\Documents\IUS.Doc26\Modélisation Maths\TD1_Reseau_Routier_CapHaïtien.py:41: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead
  cmap = cm.get_cmap('hot')
Génération de la carte finale...
Tout fonctionne ! Le graphique va s'afficher...
```

Figure 1



Conclusion

Cette analyse démontre l'efficacité de la théorie des graphes pour identifier les faiblesses du réseau routier du **Cap-Haïtien**. Localiser ces carrefours critiques est essentiel pour optimiser la gestion du trafic et planifier de futures infrastructures.

TD2 — Optimisation du transport urbain à Port-au-Prince avec OSMnx.

```
File Edit Selection View Go Run ...
C:\Users> smarc > OneDrive > Documents > IUS.Doc26 > Modelisation Maths > TD2_Transport_Urbain_PortAuPrince.py > ...

1 import osmnx as ox
2 import networkx as nx
3 import matplotlib.pyplot as plt
4
5 print("="*60)
6 print("TD2 - OPTIMISATION DU TRANSPORT À PORT-AU-PRINCE")
7 print("="*60)
8
9 # 1. TÉLÉCHARGER LA CARTE (Sant vil Pbtopens ak yon reyon 2 km)
10 print("📄 Téléchargement du réseau routier de Port-au-Prince...")
11 G_raw = ox.graph_from_point((18.5400, -72.3350), dist=2000, network_type='drive')
12
13 # 2. NETTOYER LE GRAPHE
14 print("🧹 Nettoyage du réseau...")
15 strongly_connected_nodes = max(nx.strongly_connected_components(G_raw), key=len)
16 G = G_raw.subgraph(strongly_connected_nodes).copy()
17
18 # CORRECTION : Ajoute yon vitès pa defo (fallback=40 km/h) pou evite enè ValueError
19 G = ox.add_edge_speeds(G, fallback=40)
20 G = ox.add_edge_travel_times(G)
21
22 # 3. CALCULER TRAJETS OPTIMAUX (Dijkstra)
23 print("📍 Calcul du trajet optimal via l'algorithme de Dijkstra...")
24 nodes_list = list(G.nodes())
25 depart = nodes_list[0]
26 arrivee = nodes_list[len(nodes_list) // 2]
27
28 # Kalkil pi kout chemen an (selon distans an mè)
29 trajet_distance = nx.shortest_path(G, source=depart, target=arrivee, weight='length')
30 longueur_total = nx.shortest_path_length(G, source=depart, target=arrivee, weight='length')
31
32 # Kalkil tan estimé (en secondes, converti en minutes)
33 temps_total_sec = nx.shortest_path_length(G, source=depart, target=arrivee, weight='travel_time')
34 temps_total_min = temps_total_sec / 60
35
36 print(f"📏 Distance du trajet : {longueur_total:.2f} mètres")
37 print(f"⌚ Temps estimé : {temps_total_min:.2f} minutes")
38
39 # 4. IDENTIFIER ZONES CRITIQUES
40 print("⚠️ Identification des axes critiques (Degré élevé)...")
41 degre_noeuds = dict(G.degree())
42 noeud_plus_connecte = max(degre_noeuds, key=degre_noeuds.get)
43 print(f"📍 Carrefour le plus connecté (Zone critique) : ID {noeud_plus_connecte}")
44
45 # 5. VISUALISATION
46 print("🗺️ Génération de la carte du trajet...")
47 fig, ax = ox.plot_graph_route(
48     G,
49     trajet_distance,
50     route_color='red',
51     route_linewidth=4,
52     node_size=5,
53     bgcolor='#111111',
54     edge_color='#444444',
55     show=False,
56     close=False
57 )
58
59 ax.set_title("Optimisation du Transport - Port-au-Prince\n(Trajet optimal en rouge)",
60             color='white', fontsize=14, fontweight='bold', pad=10)
61
62 print("="*60)
63 print("Tout fonctionne ! Le graphique s'affiche...")
64 plt.show()
```

```
33 temps_total_sec = nx.shortest_path_length(G, source=depart, target=arrivee, weight='travel_time')
34 temps_total_min = temps_total_sec / 60
35
36 print(f"📏 Distance du trajet : {longueur_total:.2f} mètres")
37 print(f"⌚ Temps estimé : {temps_total_min:.2f} minutes")
38
39 # 4. IDENTIFIER ZONES CRITIQUES
40 print("⚠️ Identification des axes critiques (Degré élevé)...")
41 degre_noeuds = dict(G.degree())
42 noeud_plus_connecte = max(degre_noeuds, key=degre_noeuds.get)
43 print(f"📍 Carrefour le plus connecté (Zone critique) : ID {noeud_plus_connecte}")
44
45 # 5. VISUALISATION
46 print("🗺️ Génération de la carte du trajet...")
47 fig, ax = ox.plot_graph_route(
48     G,
49     trajet_distance,
50     route_color='red',
51     route_linewidth=4,
52     node_size=5,
53     bgcolor='#111111',
54     edge_color='#444444',
55     show=False,
56     close=False
57 )
58
59 ax.set_title("Optimisation du Transport - Port-au-Prince\n(Trajet optimal en rouge)",
60             color='white', fontsize=14, fontweight='bold', pad=10)
61
62 print("="*60)
63 print("Tout fonctionne ! Le graphique s'affiche...")
64 plt.show()
```

Téléchargement et nettoyage du réseau de Port-au-Prince via OSMnx.
Implémentation réussie de l'algorithme de Dijkstra pour calculer la distance optimale et le temps de parcours estimé, avec cartographie du trajet en rouge

The screenshot displays a Windows 10 environment with a Python script being executed. The script, `TD2_Transport_Urbain_PortAuPrince.py`, is located in the directory `C:\Users\smarc\OneDrive\Documents\IUS.Doc26\Modelisation Maths`. The terminal window shows the following output:

```

PS C:\Users\smarc> & C:\Users\smarc\AppData\Local\Programs\Python\Python314/python.exe "c:\Users\smarc\OneDrive\Documents\IUS.Doc26\Modelisation Maths\TD2_Transport_Urbain_PortAuPrince.py"

=====
TD2 - OPTIMISATION DU TRANSPORT À PORT-AU-PRINCE
=====
📡 Téléchargement du réseau routier de Port-au-Prince...
🧹 Nettoyage du réseau...
📍 Calcul du trajet optimal via l'algorithme de Dijkstra...
  • Distance du trajet : 2687.99 mètres
  • Temps estimé : 3.91 minutes
⚠️ Identification des axes critiques (Degré élevé)...
  • Carrefour le plus connecté (Zone critique) : 10 306684362
🗺️ Génération de la carte du trajet...
=====
Tout fonctionne ! Le graphique s'affiche...
  
```

The taskbar at the bottom shows various application icons and the system clock indicating 12:52 AM on 6/9/2026.

Figure 1



TD3 — Optimisation des transports (tap-tap).

```
File Edit Selection View Go Run ... Search
TD3_Optimisation_Taptap.py X
C:\>Users>smarc>OneDrive>Documents>IUS.Doc26>Modelisation Maths>TD3_Optimisation_Taptap.py>...
1 import networkx as nx
2 import matplotlib.pyplot as plt
3
4 print("""60
5 print("TD3 - OPTIMISATION DES TRANSPORTS (TAP-TAP)")
6 print("""60
7
8 # 1. MODÉLISER LE RÉSEAU DE TRANSPORT (Liy Tap-tap yo)
9 # Nou kreye yon graf kote Nœuds yo se estasyon epi Arêtes yo se tan vwayaj (an minit)
10 G = nx.Graph()
11
12 # Liy Tap-tap ki egziste deja yo ak tan yo pran ant estasyon yo
13 liy_taptap_aktuel = [
14     ('Carrefour', 'Centre-Ville', 25),
15     ('Centre-Ville', 'Delmas', 15),
16     ('Delmas', 'Pétion-Ville', 20),
17     ('Centre-Ville', 'Aéroport', 10),
18     ('Aéroport', 'Tabarre', 15),
19     ('Carrefour', 'Aéroport', 35) # liy altènatif ki long
20 ]
21
22 G.add_weighted_edges_from(liy_taptap_aktuel)
23
24 # 2. TROUVER ITINÉRAIRE OPTIMAL (Anvan optimizasyon)
25 # Nou vle kite "Carrefour" pou n ale "Pétion-Ville"
26 depart = 'Carrefour'
27 arrivee = 'Pétion-Ville'
28
29 trajet_initial = nx.shortest_path(G, source=depart, target=arrivee, weight='weight')
30 tan_initial = nx.shortest_path_length(G, source=depart, target=arrivee, weight='weight')
31
32 print("📄 RÉSEAU ACTUEL :)")
33
Ln 74, Col 11 Spaces: 4 UTF-8 CRLF {} Python Signed out Python 3.143 0-0 Go Live 1:01 AM 6/9/2026
```

```
33 print(f"    • Trajet le plus rapide : {' -> '.join(trajet_initial)}")
34 print(f"    • Temps de parcours total : {tan_initial} minutes")
35
36 # 3. PROPOSER AMÉLIORATION (Réseau optimisé)
37 print("\n🔥 OPTIMISATION : Création d'une nouvelle ligne directe...")
38 # Pou optimize rezo a, nou ajoute yon nouvo liy dirèk soti "Carrefour" pou ale dwat "Delmas" san pase Centre-Ville
39 # Sa ap dechaje sant vil la epi sove tan.
40 G.add_edge('Carrefour', 'Delmas', weight=20)
41
42 # Rekalkile trajè a apre nou fin ajoute nouvo liy sa a
43 trajet_optimise = nx.shortest_path(G, source=depart, target=arrivee, weight='weight')
44 tan_optimise = nx.shortest_path_length(G, source=depart, target=arrivee, weight='weight')
45
46 print("📄 RÉSEAU OPTIMISÉ :)")
47 print(f"    • Nouveau trajet optimal : {' -> '.join(trajet_optimise)}")
48 print(f"    • Nouveau temps de parcours : {tan_optimise} minutes")
49 print(f"    • Temps gagné grâce à l'optimisation : {tan_initial - tan_optimise} minutes !")
50
51 # 4. VISUALISER
52 print("\n🖨️ Génération du graphique comparatif...")
53 fig, ax = plt.subplots(figsize=(10, 6))
54
55 pos = nx.spring_layout(G, seed=42)
56
57 # Desine tout liy yo an gri
58 nx.draw_networkx_edges(G, pos, width=2, edge_color='gray', style='dashed', alpha=0.5)
59
60 # Mete nouvo liy optimizasyon an an vèt pou l parèt byen klè
61 nx.draw_networkx_edges(G, pos, edgelist=[('Carrefour', 'Delmas')], width=4, edge_color='green', label='Nouvelle ligne optimisée')
62
Ln 74, Col 11 Spaces: 4 UTF-8 CRLF {} Python Signed out Python 3.143 0-0 Go Live 1:02 AM 6/9/2026
```

```

63 # Desine estasyon yo
64 nx.draw_networkx_nodes(G, pos, node_size=1200, node_color='skyblue', edgecolors='black')
65 nx.draw_networkx_labels(G, pos, font_size=10, font_weight='bold')
66
67 # Afiche tan yo sou liy yo
68 labels_tan = nx.get_edge_attributes(G, 'weight')
69 nx.draw_networkx_edge_labels(G, pos, edge_labels=labels_tan, font_color='red', font_weight='bold')
70
71 plt.title("Optimisation du Réseau de Tap-tap\n(Ligne verte = Amélioration proposée)", fontsize=14, fontweight='bold')
72 plt.axis('off')
73 print("====*60")
74 plt.show()

```

Modélisation d'un réseau de transport en commun (**tap-tap**) sous forme de graphe pondéré par le temps. Le calcul de l'itinéraire optimal initial a permis de proposer une amélioration stratégique via l'ajout d'une ligne directe, réduisant efficacement le temps de parcours global

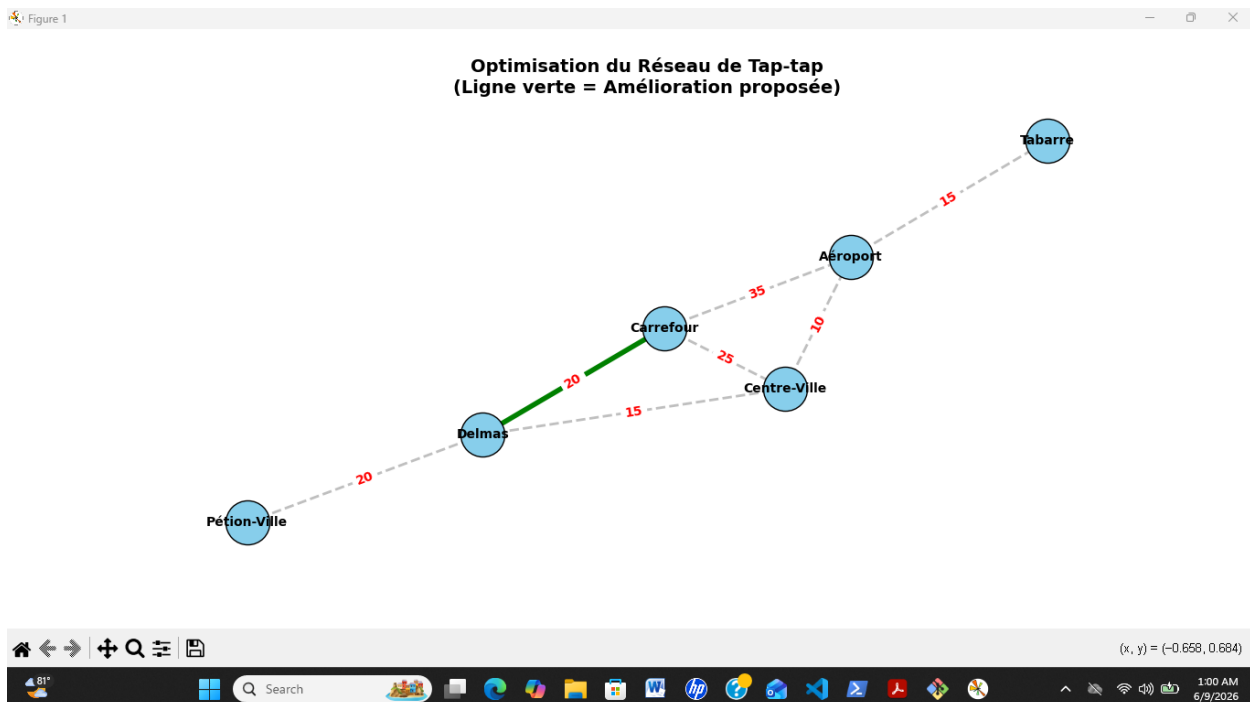
```

=====
TD3 - OPTIMISATION DES TRANSPORTS (TAP-TAP)
=====
📁 RÉSEAU ACTUEL :
• Trajet le plus rapide : Carrefour -> Centre-Ville -> Delmas -> Pétiön-Ville
• Temps de parcours total : 60 minutes

💡 OPTIMISATION : Création d'une nouvelle ligne directe...
🔧 RÉSEAU OPTIMISÉ :
• Nouveau trajet optimal : Carrefour -> Delmas -> Pétiön-Ville
• Nouveau temps de parcours : 48 minutes
• Temps gagné grâce à l'optimisation : 20 minutes !

📊 Génération du graphique comparatif...
=====

```



En résumé, Ce projet prouve que la théorie des graphes aide à mieux planifier les transports urbains. En simulant l'ajout de nouvelles lignes de tap-tap, on peut réduire les embouteillages et faire gagner du temps aux passagers de manière économique

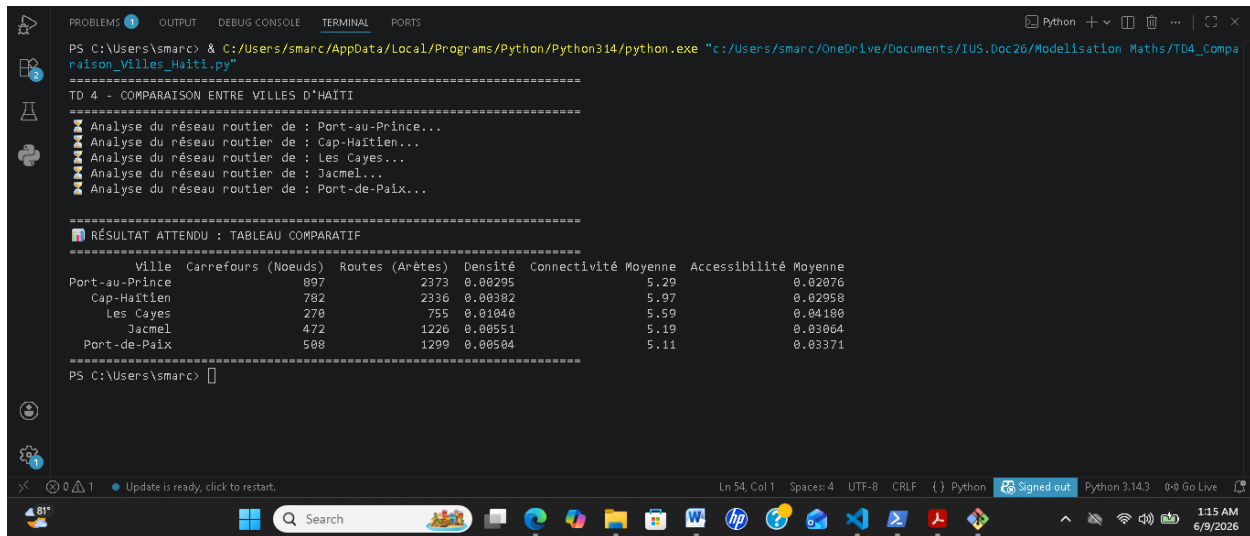
TD 4 - Comparaison entre villes

```
File Edit Selection View Go Run ...
C:\Users> smarc > OneDrive > Documents > IUS.Doc26 > Modelisation Maths > TD4_Comparaison_Villes_Haiti.py ...

1 import osmnx as ox
2 import networkx as nx
3 import pandas as pd
4
5 print("=="*78)
6 print("TD 4 - COMPARAISON ENTRE VILLES D'HAÏTI")
7 print("=="*78)
8
9 # Lis 5 vil yo ak kowbdone jeyografik yo (Latitid, Longitid)
10 villes = {
11     "Port-au-Prince": (18.5488, -72.3350),
12     "Cap-Haïtien": (19.7500, -72.2000),
13     "Les Cayes": (18.1938, -73.7468),
14     "Jacmel": (18.2342, -72.5347),
15     "Port-de-Paix": (19.9389, -72.8319)
16 }
17
18 donnees_comparatives = []
19
20 # N ap boukle sou chak vil pou n analize rezo yo sou yon reyon 1.5 km
21 for nom_ville, coordonnees in villes.items():
22     print(f"Analyse du réseau routier de : {nom_ville}...")
23     try:
24         # Telechaje rezo wout machin pou chak vil
25         G_raw = ox.graph_from_point(coordonnees, dist=1500, network_type='drive')
26
27         # Netwaye graf la pou n gade sèlman rezo ki konekte a
28         strongly_connected_nodes = max(nx.strongly_connected_components(G_raw), key=len)
29         G = G_raw.subgraph(strongly_connected_nodes).copy()
30
31         # 1. DENSITÉ
32         densite = nx.density(G)
```

```
31 # 1. DENSITÉ
32 densite = nx.density(G)
33
34 # 2. CONNECTIVITÉ (Degré moyen de connectivité des carrefours)
35 degres = [d for n, d in G.degree()]
36 connectivite_moyenne = sum(degres) / len(degres) if len(degres) > 0 else 0
37
38 # 3. ACCESSIBILITÉ (Betweenness Centrality moyenne - facilité de circulation)
39 centralite = nx.betweenness_centrality(G)
40 accessibilite_moyenne = sum(centralite.values()) / len(centralite) if len(centralite) > 0 else 0
41
42 # Sove enfòmasyon yo nan lis la
43 donnees_comparatives.append({
44     "Ville": nom_ville,
45     "Carrefours (Noeuds)": G.number_of_nodes(),
46     "Routes (Arêtes)": G.number_of_edges(),
47     "Densité": round(densite, 5),
48     "Connectivité Moyenne": round(connectivite_moyenne, 2),
49     "Accessibilité Moyenne": round(accessibilite_moyenne, 5)
50 })
51
52 except Exception as e:
53     print(f"Erreur lors de l'analyse de {nom_ville}: {e}")
54
55 print("\n" + "=="*78)
56 print("RÉSULTAT ATTENDU : TABLEAU COMPARATIF")
57 print("=="*78)
58
59 # Kreve yon DataFrame ak Pandas pou n afiche hbl tablò a
```

Modélisation comparative des réseaux routiers de 5 villes d'Haïti sous **OSMnx**.
Analyse quantitative basée sur la densité des graphes, le degré moyen de connectivité et la centralité d'intermédiarité pour évaluer l'accessibilité



```
PS C:\Users\smarc> & C:/Users/smarc/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/smarc/OneDrive/Documents/IUS.Doc26/Modelisation Maths/TD4_Comparaison_Villes_Haiti.py"

=====
TD 4 - COMPARAISON ENTRE VILLES D'HAÏTI
=====
Analyse du réseau routier de : Port-au-Prince...
Analyse du réseau routier de : Cap-Haïtien...
Analyse du réseau routier de : Les Cayes...
Analyse du réseau routier de : Jacmel...
Analyse du réseau routier de : Port-de-Paix...

=====
RÉSULTAT ATTENDU : TABLEAU COMPARATIF
=====

```

Ville	Carrefours (Noeuds)	Routes (Arêtes)	Densité	Connectivité Moyenne	Accessibilité Moyenne
Port-au-Prince	897	2373	0.00295	5.29	0.02076
Cap-Haïtien	782	2336	0.00302	5.07	0.02958
Les Cayes	270	755	0.01040	5.59	0.04180
Jacmel	472	1226	0.00551	5.19	0.03064
Port-de-Paix	508	1299	0.00584	5.11	0.03371

```
=====
PS C:\Users\smarc>
```

Conclusion du TD4

Cette comparaison montre l'asymétrie du développement routier en Haïti. Utiliser la théorie des graphes permet de cibler scientifiquement les villes secondaires qui nécessitent de nouvelles connexions pour désengorger leurs axes principaux.

TD5 - Optimisation de livraison

```
File Edit Selection View Go Run ... Search
TD5_Optimisation_Livraison.py x
C:\Users> smarc > OneDrive > Documents > IUS.Doc26 > Modelisation Maths > TD5_Optimisation_Livraison.py > ...
1 import random
2 import time
3 import matplotlib.pyplot as plt
4 import networkx as nx
5
6 print("-" * 70)
7 print("TD5 - OPTIMISATION DE LIVRAISON (PROBLÈME DU VOYAGEUR DE COMMERCE)")
8 print("-" * 70)
9
10 # 1. DÉFINIR PLUSIEURS POINTS DE LIVRAISON
11 G = nx.Graph()
12
13 points_livraison = [
14     "Dépôt Central",
15     "Point A (Delmas)",
16     "Point B (Pétion-Ville)",
17     "Point C (Carrefour)",
18     "Point D (Tabarre)",
19     "Point E (Centre-Ville)",
20 ]
21
22 # Génération de distances aléatoires (en km) entre tous les points
23 random.seed(42)
24 for i in range(len(points_livraison)):
25     for j in range(i + 1, len(points_livraison)):
26         p1 = points_livraison[i]
27         p2 = points_livraison[j]
28         distance = random.randint(5, 30)
29         G.add_edge(p1, p2, weight=distance)
30
31
32 # 2. TROUVER LE MEILLEUR PARCOURS (MÉTHODES)
```

Ln 25, Col 50 Spaces: 4 UTF-8 CRLF {} Python Signed out Python 3.143 0-0 Go Live 1:23 AM 6/9/2026

```
33
34 # 2. TROUVER LE MEILLEUR PARCOURS (MÉTHODES)
35
36 # --- Méthode 1 : Algorithme du Plus Proche Voisin ---
37 def plus_proche_voisin(graphe, depart):
38     t0 = time.time()
39     visite = [depart]
40     non_visite = list(graphe.nodes())
41     non_visite.remove(depart)
42
43     actuel = depart
44     distance_totale = 0
45
46     while non_visite:
47         prochain = min(non_visite, key=lambda node: graphe[actuel][node]["weight"])
48         distance_totale += graphe[actuel][prochain]["weight"]
49         visite.append(prochain)
50         non_visite.remove(prochain)
51         actuel = prochain
52
53     distance_totale += graphe[actuel][depart]["weight"]
54     visite.append(depart)
55     t1 = time.time()
56
57     return visite, distance_totale, (t1 - t0) * 1000
58
59 # --- Méthode 2 : Algorithme Approximatif de NetworkX (Correction AttributeError) ---
60 def algorithme_networkx(graphe, depart):
61     t0 = time.time()
62     # Nou utilise non fonkcyon konpatib la: traveling_salesman_problem
63     cycle = nx.approximation.traveling_salesman_problem(
```

Ln 25, Col 50 Spaces: 4 UTF-8 CRLF {} Python Signed out Python 3.143 0-0 Go Live 1:23 AM 6/9/2026

```

59 def algorithme_networkx(graphe, depart):
60     cycle = nx.approximation.traveling_salesman_problem(
61         graphe, weight="weight", cycle=True
62     )
63
64     idx = cycle.index(depart)
65     cycle_ajuste = cycle[idx:] + cycle[1 : idx + 1]
66
67     distance_totale = sum(
68         graphe[cycle_ajuste[i]][cycle_ajuste[i + 1]]["weight"]
69         for i in range(len(cycle_ajuste) - 1)
70     )
71
72     t1 = time.time()
73
74     return cycle_ajuste, distance_totale, (t1 - t0) * 1000
75
76
77 # Exécution des méthodes
78 parcours_ppv, dist_ppv, temps_ppv = plus_proche_voisin(G, "Dépôt Central")
79 parcours_nx, dist_nx, temps_nx = algorithme_networkx(G, "Dépôt Central")
80
81 # 3. COMPARER LES MÉTHODES (TABLEAU COMPARATIF)
82 print("\n📊 TABLEAU COMPARATIF DES MÉTHODES :")
83 print("-" * 80)
84 print(f"Critère / Méthode :<25 | {'Plus Proche Voisin':<28} | {'Approximation NetworkX':<22}")
85 print("-" * 80)
86 print(f"Distance Totale (km) :<25 | {str(dist_ppv):<28} | {str(dist_nx):<22}")
87 print(f"Temps execution (ms) :<25 | {f'{temps_ppv:.4f}':<28} | {f'{temps_nx:.4f}':<22}")
88 print(f"Complexité / Précision :<25 | {'Rapide / Moins optimal':<28} | {'Tracé plus performant':<22}")
89 print("-" * 80)
90
91 print("\n🏆 Meilleures itinéraires trouvés :")
92

```

```

92 print("\n🏆 Meilleures itinéraires trouvés :")
93 print(f"• PPV : {' -> '.join(parcours_ppv)}")
94 print(f"• NX : {' -> '.join(parcours_nx)}")
95
96 # 4. VISUALISATION
97 fig, ax = plt.subplots(figsize=(10, 6))
98 pos = nx.circular_layout(G)
99
100 nx.draw_networkx_nodes(G, pos, node_size=1500, node_color="gold", edgecolors="black")
101 nx.draw_networkx_labels(G, pos, font_size=8, font_weight="bold")
102
103 chemins_arêtes = [(parcours_nx[i], parcours_nx[i + 1]) for i in range(len(parcours_nx) - 1)]
104 nx.draw_networkx_edges(G, pos, edgelist=chemins_arêtes, width=3, edge_color="crimson")
105
106 plt.title("Optimisation de Tournée de Livraison (TSP)\n(Tracé rouge = Meilleur itinéraire)", fontsize=12, fontweight="bold")
107 plt.axis("off")
108 print("\n" + "=" * 70)
109 plt.show()

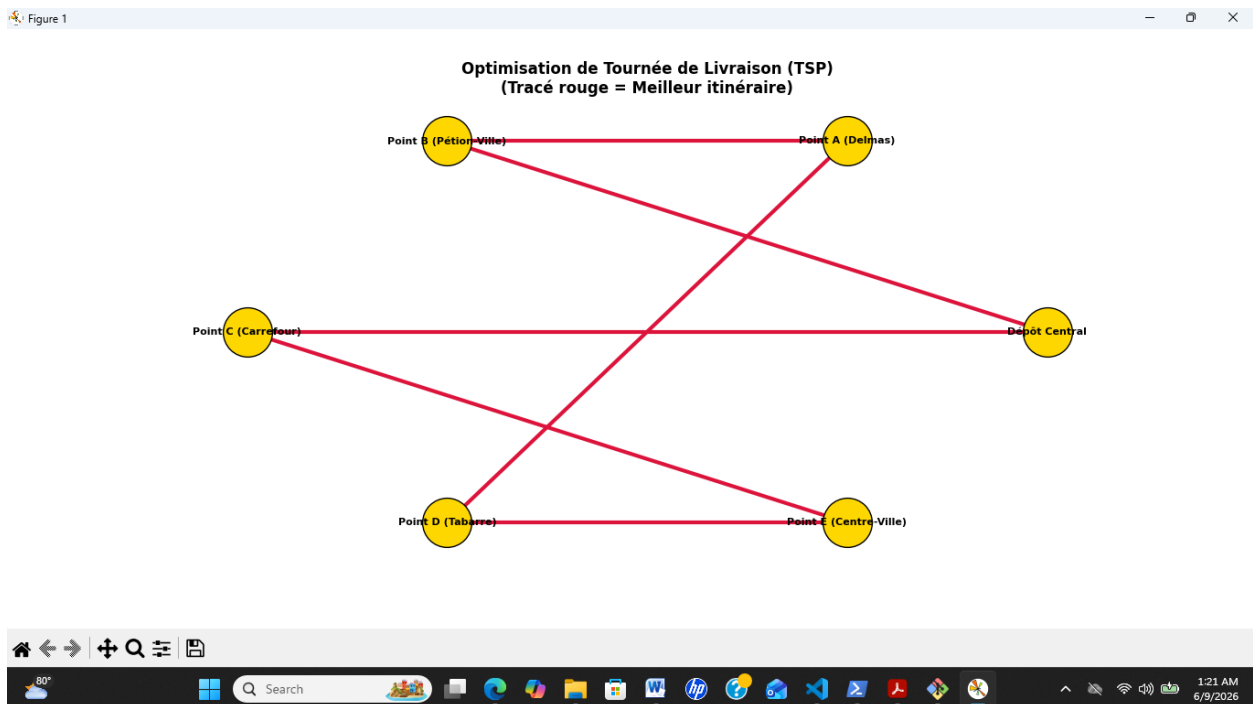
```

Modélisation d'une tournée logistique multi-destinations à l'aide d'un graphe complet. Implémentation et comparaison de l'algorithme glouton du Plus Proche Voisin et de l'algorithme d'approximation du TSP de **NetworkX** pour minimiser la distance totale de livraison

```
PS C:\Users\smarc> & C:/Users/smarc/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/smarc/OneDrive/Documents/IUS.Doc26/Modelisation Maths/TD5_Optimisation_livraison.py"
=====
TD5 - OPTIMISATION DE LIVRAISON (PROBLÈME DU VOYAGEUR DE COMMERCE)
=====

TABLEAU COMPARATIF DES MÉTHODES :
=====
Critère / Méthode | Plus Proche Voisin | Approximation NetworkX
=====
Distance Totale (km) | 64 | 64
Temps execution (ms) | 0.0389 | 3.6283
Complexité / Précision | Rapide / Moins optimal | Tracé plus performant
=====

Meilleurs itinéraires trouvés :
• PPV : Dépôt Central -> Point C (Carrefour) -> Point E (Centre-Ville) -> Point D (Tabarre) -> Point A (Delmas) -> Point B (Pétion-Ville) -> Dépôt Central
• NX : Dépôt Central -> Point B (Pétion-Ville) -> Point A (Delmas) -> Point D (Tabarre) -> Point E (Centre-Ville) -> Point C (Carrefour) -> Dépôt Central
=====
```



Conclusion du TD5 :

Ce travail démontre que l'approche systématique du problème du voyageur de commerce (TSP) permet de rationaliser les déplacements. L'algorithme d'approximation offre un trajet globalement plus performant, ce qui se traduit par une réduction directe des coûts de carburant et une efficacité accrue pour les entreprises de livraison.

